

CURSO DE PYTHON

**Corporación Ecuatoriana para el Desarrollo de la
Investigación y la Academia**

Ing. María Fernanda Granda J., PhD

13-07-2026

**“Conectando ideas,
transformando
sociedades”**

Curso: Python Intermedio – Avanzado: Desarrollo de Soluciones, Automatización e Inteligencia Artificial

Competencias

- Desarrollar aplicaciones escalables mediante programación orientada a objetos, buenas prácticas de desarrollo y estructuras avanzadas de Python.
- Procesar, analizar y visualizar datos utilizando bibliotecas especializadas para apoyar la toma de decisiones en contextos de ingeniería e investigación.
- Diseñar soluciones de automatización e integrar servicios web e inteligencia artificial para optimizar procesos y generar valor en entornos profesionales.

¿De qué se trata el Curso?

- **Ciencias de Datos:** Convertir datos en información útil para tomar mejores decisiones
- **Desarrollo de soluciones (Pensamiento Computacional):** Resolver problemas paso a paso: descomponer, hallar patrones y diseñar una solución
- **Inteligencia artificial (Vibe coding):** Programar con ayuda de la IA, escribiéndole lo que se quiere lograr.

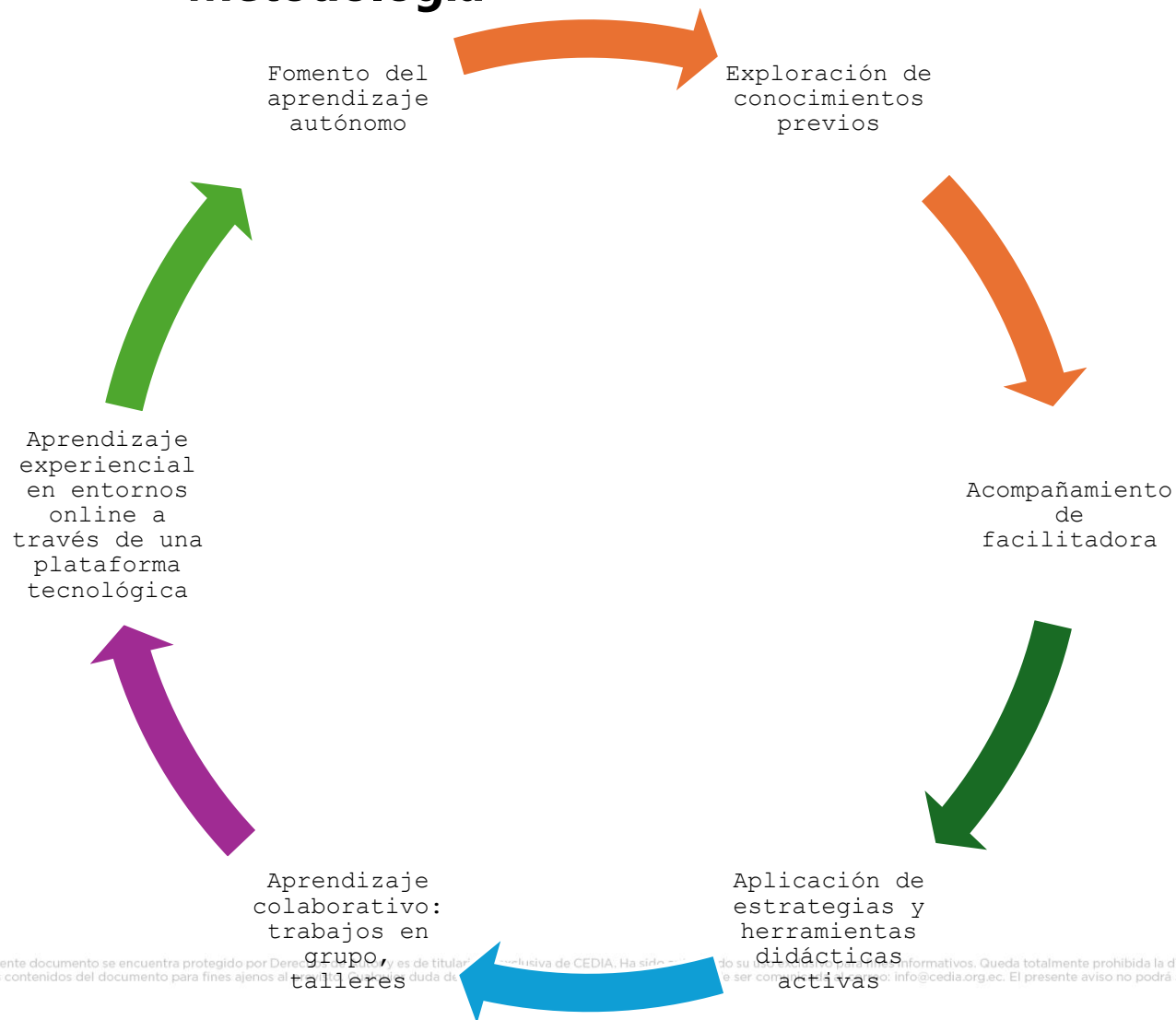
Recursos

Recursos recomendados:

- Documentación oficial de Python sobre funciones
- Visual Studio Code + extensión Python
- Repositorio GitHub / drive para subir archivos (opcional)

Con el fin de potenciar el aprendizaje del participante aplicamos las siguientes metodologías de trabajo basadas en el enfoque de Aprender-haciendo:

Metodología



Tema 1: Python Intermedio y Buenas Prácticas

- Diagnóstico y Repaso
- Funciones Avanzadas
- Expresiones Lambda
- Comprensiones de Listas y Diccionarios
- Actividades en Clase

Tema 1: Python Intermedio y Buenas Prácticas

Objetivo:

Al finalizar la clase, el participante será capaz de utilizar funciones avanzadas, expresiones lambda y comprensiones de listas y diccionarios para desarrollar soluciones más eficientes, legibles y reutilizables siguiendo buenas prácticas de programación.

Instalar Python

Para configurar y usar Python en Visual Studio Code (VS Code):

- Instalar el intérprete de Python en tu sistema operativo
 - Visitar [Python.org](https://python.org) y descargar la versión más reciente recomendada para tu sistema operativo (Windows, Mac o Linux)
- Instalar la extensión oficial dentro del editor.

Diagnóstico y Repaso

Actividad:

La evaluación diagnóstica se realizará contestando un cuestionario en el aula virtual en Moodle.

Tiempo: 10 minutos

Python: El Lenguaje

¿Qué es?

Una forma de dar instrucciones al computador escribiendo texto parecido al inglés. Es el lenguaje más usado en ciencia de datos.

¿Cómo funciona?

Escribes código, el computador lo ejecuta y devuelve un resultado. Como una receta de cocina: pasos en orden, uno por uno.

¿Por qué Python?

Es fácil de leer y miles de herramientas gratuitas (librerías) para trabajar con datos.

Diagnóstico y Repaso

Actividad:

¿Qué hace el código?

¿Cómo podríamos reutilizarlo?

¿Qué pasaría si tenemos varias listas?.

```
</> Python
```

```
notas = [8, 9, 7, 10, 6]
```

```
suma = 0
```

```
for nota in notas:  
    suma += nota
```

```
promedio = suma / len(notas)
```

```
print(promedio)
```

Funciones

Conceptos:

- Son bloques de código independientes diseñados para realizar una tarea específica.
- Se definen mediante la palabra clave `def` y permiten evitar la duplicación de código, facilitando el mantenimiento y mejorando la legibilidad del proyecto.

Funciones Avanzadas

Funciones reutilizables: recibe datos (parámetros), realiza operaciones y devuelve un resultado (usando return)

```
def <nombre funcion>(<parámetros>):  
    return <valor a retornar>
```

Ejemplo de Funciones

```
1  # Definición de funciones
2  def obtenerresto(num1, num2):
3      ... return num1 - num2 * (num1 // num2) # Sin usar operador %
4
5  def es_multiplo(x, y):
6      ... return obtenerresto(x, y) == 0
7
8  # Programa principal
9  a = int(input("Primer número: "))
10 b = int(input("Segundo número: "))
11
12 resto = obtenerresto(a, b)
13
14 print(f"El resto entre {a} y {b} es {resto}")
15 if es_multiplo(a, b):
16     ... print(f"{a} es múltiplo de {b}")
```

Funciones Avanzadas

Funciones con parámetros: Para crear funciones realmente versátiles, puedes definir parámetros con valores por defecto o utilizar `*args` y `**kwargs` para aceptar un número variable de argumentos:

- Valores por defecto: Permiten llamar a la función omitiendo ciertos argumentos.
- `*args`: Recibe una lista de argumentos posicionales.
- `**kwargs`: Recibe argumentos con nombre en forma de diccionario..

Parámetros por defecto

```
</> Python
def saludar(nombre="Usuario"):
    print(f"Hola {nombre}")
```

Parámetros arbitrarios (*args)

```
</> Python
def sumar(*numeros):
    return sum(numeros)

print(sumar(2,3,4,5))
```

Parámetros nombrados (**kwargs)

```
</> Python
def mostrar_datos(**datos):
    for clave, valor in datos.items():
        print(clave, valor)

mostrar_datos(nombre="Juan", edad=25)
```

Modularidad: Creación de Módulos

Para llevar la reutilización al máximo, puedes guardar tus funciones en un archivo separado (por ejemplo, `mis_utilidades.py`) y reutilizarlas en cualquier otro script de Python usando la instrucción **import**.

```
# En archivo mis_utilidades.py
def saludar(nombre):
    return f"¡Hola, {nombre}!"

# En otro archivo de tu proyecto
import mis_utilidades

mensaje = mis_utilidades.saludar("Ana")
```


Actividad 1

Crear funciones que permitan:

- Calcular salario base
- Calcular horas extras
- Calcular descuentos
- Mostrar salario final

`</>` Python

```
def calcular_salario(horas, tarifa):  
    return horas * tarifa
```

Entregable

Subir al aula virtual de CEDIA hasta el miércoles 15 de Julio 6:59 pm.

- Archivo .py
- Código documentado

Funciones Lambda (Anónimas)

Conceptos

- Las funciones lambda en Python son funciones anónimas y de una sola línea.
- En lugar de usar la palabra clave def, se definen con la palabra lambda, seguida de los parámetros, dos puntos y una única expresión de retorno.
- Son ideales para operaciones rápidas donde la función solo se usará una vez
- La estructura general de una función lambda es:

```
lambda <parámetros>: <valor a retornar>
```

```
def sumar(x, y):  
    return x + y
```

```
sumar = lambda x, y: x + y  
print(sumar(3, 5)) # Salida: 8
```

Funciones Lambda

¿Cuándo se deben utilizar?

Se usan con frecuencia junto con funciones integradas como `map()`, `filter()` y `sorted()` para manipular datos rápidamente sin tener que escribir una función formal.

- **Con `map()`:** Para aplicar una operación a cada elemento de una lista.

```
numeros = [1,2,3,4]
doble = list (map (lambda x: x*2, numeros))
print (doble)
#salida: [2, 4, 6, 8]
```

Funciones Lambda

¿Cuándo se deben utilizar?

- **Con filter():** Para extraer elementos que cumplen una condición..

```
numeros = [1,2,3,4, 5, 6]
pares = list(filter(lambda x: x % 2==0, numeros))
print (pares)
#salida: [2, 4, 6]
```

Funciones Lambda

¿Cuándo se deben utilizar?

- **Con `sorter()`:** Para ordenar elementos usando un criterio específico.

```
puntos = [(1,2), (4,1), (5,9)]  
ordenar_por_y = sorted(puntos, key=lambda punto:punto[1])  
print(ordenar_por_y)  
#salida: [(4,1), (1,2), (5,9)]
```

Limitaciones de las Funciones Lambda

- **Una sola expresión:** Solo pueden contener una línea de código y no admiten instrucciones de flujo de control complejas (como if/else multilínea o bucles).
- **Legibilidad:** Aunque son muy útiles, abusar de ellas puede hacer que el código sea difícil de leer para otros programadores, especialmente en expresiones largas.

Actividad 2

Ranking de estudiantes:

Dado:

```
</> Python  
  
estudiantes = [  
    ("Ana", 95),  
    ("Luis", 80),  
    ("María", 100),  
    ("Pedro", 70)  
]
```

- Ordenar por nota
- Obtener aprobados (>70)
- Generar lista con nota incrementada en 5%

Entregable

Subir al aula virtual de CEDIA hasta el miércoles 15 de Julio 6:59 pm.

- Archivo .py
- Código documentado

GRACIAS